

➤

💡 WHY TREES?

Trees store data in a non-linear hierarchy (DOM elements, file systems, organizational charts). Binary Search Trees (BST) enable **O(log N) lookup, insertion, and deletion!**

💡 THE BST INORDER AHA

An **Inorder Traversal** ('left → root → right') of a Binary Search Tree ALWAYS produces elements in strictly ****sorted ascending order****!

➤

1 CORE NODE STRUCTURE

```
public class TreeNode {
    int val;
    TreeNode left;
    TreeNode right;
    TreeNode(int val) { this.val = val; }
    TreeNode(int val, TreeNode left,
        TreeNode right) {
        this.val = val; this.left = left;
        this.right = right;
    }
}
```

➤

2 VISUALIZING 3 DFS TRAVERSALS

Tree: [2] with left [1], right [3]:

```
Preorder (Root, L, R) : 2 → 1 → 3 (Copy/Serialize)
Inorder (L, Root, R) : 1 → 2 → 3 (Sorted BST!)
Postorder(L, R, Root) : 1 → 3 → 2 (Delete/Bottom-Up)
```

➤

📖 PREREQUISITES & COMPLEXITY REASONING

Traversal / Operation	Time Complexity	Auxiliary Space
Balanced BST Lookup / Insert	O(log N)	O(log N) stack
Skewed Tree (Worst Case)	O(N)	O(N) stack
Tree DFS (Pre / In / Post)	O(N)	O(H) recursion stack
Tree BFS (Level Order)	O(N)	O(W) Queue width

➤

⚡ CLUES THAT SCREAM TREE DFS / BFS

1. "Max depth / Height of tree" → Bottom-Up Postorder DFS

2. "Level order / Right side view" → BFS with Queue level size

3. "Lowest Common Ancestor" → Postorder DFS return non-null

4. "Validate BST / Kth Smallest" → Inorder DFS tracking prev node

➤

1 LEVEL ORDER BFS TEMPLATE ('TreeUtils')

```
public static List<List<Integer>> levelOrder(TreeNode root) {
    List<List<Integer>> res = new ArrayList<>();
    if (root == null) return res;
    Queue<TreeNode> q = new LinkedList<>();
    q.offer(root);
    while (!q.isEmpty()) {
        int size = q.size();
        List<Integer> level = new ArrayList<>();
        for (int i = 0; i < size; i++) {
            TreeNode curr = q.poll();
            level.add(curr.val);
            if (curr.left != null) q.offer(curr.left);
            if (curr.right != null) q.offer(curr.right);
        }
        res.add(level);
    }
    return res;
}
```

➤

2 RECURSIVE INORDER TRAVERSAL

```
public static void inorder(TreeNode root, List<Integer> res) {
    if (root == null) return;
    inorder(root.left, res);
    res.add(root.val); // Process root in middle
    inorder(root.right, res);
}
```

➤

3 BST SEARCH HELPER

```
public static TreeNode searchBST(TreeNode root, int val) {
    if (root == null || root.val == val) return root;
    if (val < root.val) return searchBST(root.left, val);
    return searchBST(root.right, val);
}
```

➤

4 MAXIMUM DEPTH / HEIGHT

```
public static int maxDepth(TreeNode root) {
    if (root == null) return 0;
    return 1 + Math.max(maxDepth(root.left), maxDepth(root.right));
}
```

>

PATTERN 1

TREE INVERSION & SYMMETRY (Preorder DFS)

e.g. Invert Binary Tree (LC 226), Same Tree (LC 100)

WHEN TO USE
Swapping left and right pointers at every node or verifying structural identity between two trees.

TEMPLATE — LC 226

```
public TreeNode invertTree(TreeNode root) {
    if (root == null) return null;
    TreeNode temp = root.left;
    root.left = invertTree(root.right);
    root.right = invertTree(temp);
    return root;
}
```

PATTERN 2

SAME TREE & SUBTREE MATCHING

e.g. Same Tree (LC 100), Subtree of Another Tree (LC 572)

WHEN TO USE
Check if tree `subRoot` matches any node in `root`.

TEMPLATE — LC 572

```
public boolean isSubtree(TreeNode root, TreeNode subRoot) {
    if (root == null) return false;
    if (isSame(root, subRoot)) return true;
    return isSubtree(root.left, subRoot) || isSubtree(root.right, subRoot);
}

private boolean isSame(TreeNode p, TreeNode q) {
    if (p == null && q == null) return true;
    if (p == null || q == null || p.val != q.val) return false;
    return isSame(p.left, q.left) && isSame(p.right, q.right);
}
```

>

PATTERN 3

VALIDATE BINARY SEARCH TREE (Min / Max Bound Range)

e.g. Validate Binary Search Tree (LC 98)

BOUNDARY PASS STRATEGY
Pass valid range `[min, max]` down recursion. For left child, range becomes `[min, root.val]`. For right child, range becomes `[root.val, max]`.

TEMPLATE — LC 98

```
public boolean isValidBST(TreeNode root) {
    return validate(root, null, null);
}

private boolean validate(TreeNode node, Integer min, Integer max) {
    if (node == null) return true;
    if ((min != null && node.val <= min) || (max != null && node.val >= max)) return false;
    return validate(node.left, min, node.val) && validate(node.right, node.val, max);
}
```

PATTERN 4

KTH SMALLEST ELEMENT IN BST (Inorder Traversal)

e.g. Kth Smallest Element in a BST (LC 230)

INORDER STACK MECHANICS
Use explicit stack to simulate inorder traversal. The K-th popped element is guaranteed to be the K-th smallest value!

TEMPLATE — LC 230

```
public int kthSmallest(TreeNode root, int k) {
    Deque<TreeNode> stack = new ArrayDeque<>();
    TreeNode curr = root;
    while (curr != null || !stack.isEmpty()) {
        while (curr != null) { stack.push(curr); curr = curr.left; }
        curr = stack.pop();
        if (--k == 0) return curr.val;
        curr = curr.right;
    }
    return -1;
}
```

>

PATTERN5

LOWEST COMMON ANCESTOR (LCA in BST & Binary Tree)

e.g. LCA of a BST (LC 235), LCA of a Binary Tree (LC 236)

LCA LOGIC

For BST: If both $p, q < \text{root}$, go left. If both $> \text{root}$, go right. Otherwise current node is split point (LCA)!

TEMPLATE — LC 235

```
public TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode q) {
    if (p.val < root.val && q.val < root.val) return lowestCommonAncestor(root.left, p, q);
    if (p.val > root.val && q.val > root.val) return lowestCommonAncestor(root.right, p, q);
    return root;
}
```

PATTERN6

RIGHT SIDE VIEW (Level Order BFS Last Element)

e.g. Binary Tree Right Side View (LC 199)

BFS LAST ELEMENT STRATEGY

Run level order BFS. When loop reaches last element of current level ($i == \text{size} - 1$), record node value in result list!

TEMPLATE — LC 199

```
public List<Integer> rightSideView(TreeNode root) {
    List<Integer> res = new ArrayList<>();
    if (root == null) return res;
    Queue<TreeNode> q = new LinkedList<>(); q.offer(root);
    while (!q.isEmpty()) {
        int size = q.size();
        for (int i = 0; i < size; i++) {
            TreeNode curr = q.poll();
            if (i == size - 1) res.add(curr.val);
            if (curr.left != null) q.offer(curr.left);
            if (curr.right != null) q.offer(curr.right);
        }
    }
    return res;
}
```

>

PATTERN7

MAXIMUM PATH SUM (Postorder Global Max State)

e.g. Binary Tree Maximum Path Sum (LC 124)

STATE UPDATE FORMULA

At each node, max path through node $= \text{node.val} + \max(0, \text{left}) + \max(0, \text{right})$. Update global max. Return max single branch $\text{node.val} + \max(0, \max(\text{left}, \text{right}))$.

TEMPLATE — LC 124

```
private int maxPath = Integer.MIN_VALUE;
public int maxPathSum(TreeNode root) {
    gain(root); return maxPath;
}
private int gain(TreeNode node) {
    if (node == null) return 0;
    int leftGain = Math.max(0, gain(node.left));
    int rightGain = Math.max(0, gain(node.right));
    maxPath = Math.max(maxPath, node.val + leftGain + rightGain);
    return node.val + Math.max(leftGain, rightGain);
}
```

PATTERN8

CONSTRUCT TREE FROM PREORDER & INORDER (HashMap Index Map)

e.g. Construct Binary Tree from Preorder and Inorder Traversal (LC 105)

CONSTRUCTION LOGIC

Preorder first element is root. Inorder divides tree into left subtree and right subtree! Map inorder values to indices for $O(1)$ lookup.

TEMPLATE — LC 105

```
private Map<Integer, Integer> inMap = new HashMap<>();
private int preIdx = 0;
public TreeNode buildTree(int[] preorder, int[] inorder) {
    for (int i = 0; i < inorder.length; i++) inMap.put(inorder[i], i);
    return build(preorder, 0, inorder.length - 1);
}
```

TREES & BST — WHICH PATTERN TO USE?

Start: What is the tree problem goal?

Invert / Swap left & right

→

Preorder DFS

Swap pointers recursively

Height / Max Path Sum

→

Postorder DFS

Bottom-up left/right result

Validate BST / Kth Smallest

→

Inorder Traversal

Produces sorted values

Level-by-Level / Right View

→

BFS Queue

Process level size

Shared Ancestor of 2 Nodes

→

Lowest Common Ancestor

BST val check / Postorder

TRIGGER WORDS DIRECTORY

Trigger Word	Variant	Action
"Sorted order in BST"	Inorder Traversal	`left → root → right`
"Level order", "Right view"	BFS Queue	Loop `i = 0..size-1`
"Lowest common ancestor"	LCA Pattern	Compare p, q to root
"Max path sum"	Postorder DFS	Return $\text{node} + \max(\text{L}, \text{R})$

INTERVIEW TRADE-OFFS & FOLLOW-UPS

Interviewer Asks	Your Answer
DFS vs BFS memory on trees?	DFS stack memory is $O(H)$ (tree height). BFS queue memory is $O(W)$ (max level width)!
Why does BST Inorder sort?	By definition, all left children are smaller than root, and all right children are larger!

EASY — Fundamental Tree DFS Operations

LC 226 · Invert Binary Tree

EASY

Pattern: Preorder DFS
Key Insight: Swap left & right recursively.

```
public TreeNode invertTree(TreeNode root) {
    if (root == null) return null;
    TreeNode tmp = root.left;
    root.left = invertTree(root.right);
    root.right = invertTree(tmp);
    return root;
}
```

LC 104 · Max Depth of Binary Tree

EASY

Pattern: Postorder DFS
Key Insight: \$1 + \max(\text{left}, \text{right})\$.

```
public int maxDepth(TreeNode root) {
    if (root == null) return 0;
    return 1 + Math.max(maxDepth(root.left), maxDepth(root.right));
}
```

LC 100 · Same Tree

EASY

Pattern: Dual Tree DFS
Key Insight: Compare 'p.val == q.val' & structural match.

```
public boolean isSameTree(TreeNode p, TreeNode q) {
    if (p == null && q == null) return true;
    if (p == null || q == null || p.val != q.val) return false;
    return isSameTree(p.left, q.left) && isSameTree(p.right, q.right);
}
```

MEDIUM — BST Validation, LCA & Right View

LC 98 · Validate BST

MEDIUM

Pattern: Min/Max Boundary DFS
Key Insight: Pass valid [min, max] range down recursion.

```
public boolean isValidBST(TreeNode root) { return val(root, null, null); }
private boolean val(TreeNode n, Integer min, Integer max) {
    if (n == null) return true;
    if ((min != null && n.val <= min) || (max != null && n.val >= max)) return false;
    return val(n.left, min, n.val) && val(n.right, n.val, max);
}
```

LC 235 · LCA of BST

MEDIUM

Pattern: BST LCA Search
Key Insight: Move left if both < root, right if > root.

```
public TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode q) {
    if (p.val < root.val && q.val < root.val) return lowestCommonAncestor(root, p, q);
    if (p.val > root.val && q.val > root.val) return lowestCommonAncestor(root, p, q);
    return root;
}
```

MEDIUM — Subtree & Level Order BFS

LC 572 · Subtree of Another Tree

EASY

Pattern: DFS Subtree Match
Key Insight: 'isSame(root, subRoot)' or check children.

```
public boolean isSubtree(TreeNode root, TreeNode subRoot) {
    if (root == null) return false;
    if (isSame(root, subRoot)) return true;
    return isSubtree(root.left, subRoot) || isSubtree(root.right, subRoot);
}
```

LC 102 · Binary Tree Level Order

MEDIUM

Pattern: Queue BFS
Key Insight: Process 'size' elements per level.

```
public List<List<Integer>> levelOrder(TreeNode root) {
    List<List<Integer>> res = new ArrayList<>();
    if (root == null) return res;
    Queue<TreeNode> q = new LinkedList<>();
    q.offer(root);
    while (!q.isEmpty()) {
        int sz = q.size();
        List<Integer> lvl = new ArrayList<>();
        for (int i=0; i<sz; i++) {
            TreeNode node = q.poll();
            lvl.add(node.val);
            if (node.left != null) q.offer(node.left);
            if (node.right != null) q.offer(node.right);
        }
        res.add(new ArrayList<Integer>(lvl));
    }
    return res;
}
```

HARD — Max Path Sum & Construction

LC 124 · Binary Tree Max Path Sum

HARD

Pattern: Postorder State Max
Key Insight: Update global max with 'node + left + right'.

```
int maxP = Integer.MIN_VALUE;
public int maxPathSum(TreeNode root) { gain(root); return maxP; }
private int gain(TreeNode n) {
    if (n == null) return 0;
    int l = Math.max(0, gain(n.left));
    int r = Math.max(0, gain(n.right));
    maxP = Math.max(maxP, n.val + l + r);
    return n.val + Math.max(l, r);
}
```

LC 230 · Kth Smallest in BST

MEDIUM

Pattern: Inorder Stack
Key Insight: K-th popped element is K-th smallest!

```
public int kthSmallest(TreeNode root, int k) {
    Deque<TreeNode> st = new ArrayDeque<>();
    TreeNode c = root;
    while (c != null || !st.isEmpty()) {
        while (c != null) st.push(c);
        c = c.left;
        c = st.pop();
        k--;
        if (k == 0) return c.val;
        c = c.right;
    }
    return -1;
}
```

> DRY RUN — Validate BST (Root [2], Left [1], Right [3])

Node	Min Bound	Max Bound	Condition Check	Result
2 (Root)	null	null	$\text{null} < 2 < \text{null}$ ↪ Valid!	Check left (1) & right (3)
1 (Left)	null	2	$\text{null} < 1 < 2$ ↪ Valid!	Returns true
3 (Right)	2	null	$2 < 3 < \text{null}$ ↪ Valid!	Returns true ↪ Tree is valid BST! ✓

✓ MASTERY CHECKLIST — CAN YOU DO THIS?

☐ Invert Binary Tree in 2m

☐ Write Level Order BFS Queue from memory

☐ Validate BST using [min, max] boundaries

☐ Code Kth Smallest in BST via Inorder

☐ Code LCA of BST in 3 minutes

☐ Implement Binary Tree Right Side View

☐ Write Binary Tree Max Path Sum (LC 124)

☐ Prove why BST lookup is $O(\log N)$

MATHEMATICAL PROOF — BALANCED BST LOOKUP

Claim: Searching in a balanced Binary Search Tree takes $O(\log N)$ time.

Proof: At each step, comparing target with `root.val` eliminates half of the remaining tree (left or right subtree). Total nodes remaining after k steps $= N / 2^k$. When $N / 2^k = 1 \Rightarrow 2^k = N \Rightarrow k = \log_2 N$
↪ **$O(\log N)$ Time!**

GOLDEN RULES — NEVER FORGET

Rule 1 — Inorder BST = Sorted Array

Inorder traversal of BST ALWAYS yields sorted values. Use this property for BST validation and K-th smallest elements!

Rule 2 — Guard Against Integer Overflow in BST Bounds

In Validate BST, use `Integer` objects (`null`) or `Long.MIN_VALUE` / `Long.MAX_VALUE` instead of `Integer.MIN_VALUE` to avoid boundary overflow!